

atelier 12 javascript

matisse

kra

bts sio

sommaire

Contexte	2
Présentation de l'environnement navigateur	2
Exploration du DOM	2
Navigation entre les nœuds DOM	3
exercice 1 réaliser :	3
exercice 2 réaliser :	5
Manipulation des tableaux HTML	5
exercice 3 réaliser :	6
Recherches dans le DOM : méthodes get et query	7
Méthodes getElementBy...	7
Méthodes querySelector et querySelectorAll	7
exercice 4 realiser:	8
Propriétés de nœud : type, balise et contenu	9
Type de nœud (nodeType)	9
Nom de la balise (nodeName / tagName)	9
Contenu du nœud	9
Intérêt des propriétés de nœud	10
exercice 5 realiser:	10
exercice 6 realiser:	11
exercice 7 realiser:	11
Attributs et propriétés du DOM	12
exercice 8 realiser	12
exercice 9 realiser	14
Modification du document	15
exercice 10 realiser:	15
exercice 11 réaliser:	16
exercice 12 réaliser:	17
exercice 13 realiser:	18
exercice 14 réaliser:	19
exercice 15 réaliser:	20
exercice 16 réaliser:	21
exercice 17 réaliser:	22
exercice 18 réaliser:	24
exercice 19 réaliser:	24
Styles et classes	26
Modification directe des styles	26
Utilisation des classes CSS	26

Exercice 20 : Créer une Notification	27
Taille des éléments et défilement	27
Mesurer la taille d'un élément	27
Contrôle du défilement	28
Exercice 21 défilement vers le bas :	28
Exercice 22 Calculer la largeur de la Scrollbar	29
Exercice 23 placer le ballon au centre du terrain	29
Exercice 24 différence largeur css vs clientwidth	30
Coordonnées des éléments	30
Propriétés principales	31
Exemples d'utilisation	31
Exercice 25 trouver les coordonnées de la fenêtre du champ	31
Conclusion	31

Contexte

Dans le cadre du cours de développement web, ce tp a pour objectif de découvrir et d'utiliser JavaScript côté navigateur. Les exercices permettent de comprendre comment JavaScript interagit avec le navigateur, la page web (DOM) et l'utilisateur.

- le rôle des objets globaux du navigateur,
- la structure du DOM (Document Object Model),
- la navigation entre les nœuds du DOM,
- l'utilisation des outils de développement du navigateur pour analyser et manipuler une page web.

Présentation de l'environnement navigateur

JavaScript s'exécute dans le navigateur via des objets globaux, en particulier l'objet window. Cet objet représente la fenêtre du navigateur et permet d'accéder à différentes fonctionnalités comme le document HTML, l'historique ou encore les dimensions de la fenêtre.

Le DOM représente la structure du document HTML sous forme d'arbre. Chaque élément HTML est un nœud que l'on peut parcourir, modifier ou sélectionner à l'aide de JavaScript.

Exploration du DOM

À l'aide des outils de développement du navigateur, la structure du DOM a été observée en temps réel.

L'onglet Éléments permet de visualiser l'arborescence HTML et de repérer les relations entre les nœuds :

- parents,

- enfants,
- frères et sœurs.

Navigation entre les nœuds DOM

Les propriétés du DOM permettent de se déplacer entre les éléments :

- accès aux enfants d'un élément,
- navigation vers le parent,
- navigation entre les éléments frères.

Ces manipulations facilitent la compréhension de la structure de la page et sont essentielles pour modifier dynamiquement le contenu.

exercice 1 réaliser :

Enfants DOM

Regardez cette page :

```
1 <html>
2 <body>
3   <div>Users:</div>
4   <ul>
5     <li>John</li>
6     <li>Pete</li>
7   </ul>
8 </body>
9 </html>
```

Pour chacun des éléments suivants, donnez au moins un moyen d'y accéder :

- Le nœud `<div>` du DOM ?
- Le nœud `<ul>` du DOM ?
- Le deuxième `<li>` (avec Pete) ?

il est important de distinguer deux façons de faire : la navigation par nœuds (relations directes) et la sélection par sélecteurs CSS (plus moderne et souvent utilisée en JS).

Voici les solutions possibles pour accéder à ces éléments via JavaScript :

1. Le nœud `<div>`

Le `<div>` est le premier enfant élément de `<body>`.

- Via la relation enfant :
JavaScript
`document.body.firstChild`
// ou

```
document.body.children[0]
```

2. Le nœud

Le est le deuxième enfant élément de <body> (ou le dernier dans ce cas précis).

- Via la relation enfant/frère :
JavaScript
document.body.lastElementChild
// ou
document.body.children[1]

3. Le deuxième (avec Pete)

Il faut d'abord atteindre le parent , puis prendre son deuxième enfant.

- Via la navigation complète (Body -> UL -> LI) :
JavaScript
- document.body.lastElementChild.lastElementChild
- Via l'index des enfants :
JavaScript
document.body.children[1].children[1]

attention à la différence entre childNodes et children (ou firstChild et firstElementChild) :

1. childNodes / firstChild : Ils comptent TOUT, y compris les sauts de ligne (l'indentation) et les espaces vides qui sont considérés comme des "nœuds de texte" (#text).
2. children / firstElementChild : Ils ne comptent que les balises HTML réelles (<div>, , etc.). C'est généralement ce que l'on veut utiliser pour éviter les bugs.

```
1  <html>
2    <body>
3      <div> User:</div>
4      <ul>
5        <li>john</li>
6        <li>pete</li>
7      </ul>
8    </body>
9  </html>
10  <script>
11    document.body.firstChild
12    document.body.lastElementChild
13    document.body.lastElementChild.lastElementChild
14  </script>
```

exercice 2 réaliser :

La question des frères et sœurs

Si `element` – est un nœud élément arbitraire du DOM ...

- Est-il vrai que `elem.lastChild.nextSibling` est toujours `null` ?
- Est-il vrai que `elem.children[0].previousSibling` est toujours `null` ?

La question des frères et sœurs1. `elem.lastChild.nextSibling` est toujours `null` ?

OUI L'explication logique : `lastChild` désigne par définition le tout dernier nœud contenu dans l'élément parent. Puisqu'il est le dernier, il ne peut rien y avoir après lui à l'intérieur de ce même parent. Son "frère suivant" (`nextSibling`) est donc forcément inexistant, c'est-à-dire `null`. Si `elem` est vide (aucun enfant), `lastChild` vaut `null`. Accéder à `nextSibling` provoquerait une erreur en JavaScript.

2. `elem.children[0].previousSibling` est toujours `null` ?

NON je m'explique `elem.children[0]` cible le premier enfant Élément (la première balise HTML, par exemple un `<div>` ou un `<li>`). Mais `previousSibling` regarde le Nœud précédent (qui peut être du texte, un commentaire ou un saut de ligne). Il est tout à fait possible d'avoir du texte ou un simple saut de ligne (un "nœud texte" vide) avant la première balise HTML.

Manipulation des tableaux HTML

Le fichier `table.html` a permis de travailler sur la manipulation des tableaux via le DOM. Les différentes parties du tableau (lignes, colonnes, cellules) ont été identifiées et sélectionnées à l'aide de JavaScript.

Un exercice consistait notamment à sélectionner certaines cellules spécifiques, comme les cellules diagonales d'un tableau.

exercice 3 réaliser :

Sélectionner toutes les cellules diagonales

Écrivez le code pour colorer toutes les cellules du tableau diagonal en rouge.

Vous devrez obtenir toutes les diagonales `<td>` de la `<table>` et les colorer en utilisant le code :

```
1 // td doit être la référence à la cellule du tableau
2 td.style.backgroundColor = 'red';
```

Le résultat devrait être :

1:1	2:1	3:1	4:1	5:1
1:2	2:2	3:2	4:2	5:2
1:3	2:3	3:3	4:3	5:3
1:4	2:4	3:4	4:4	5:4
1:5	2:5	3:5	4:5	5:5

L'API du DOM pour les tableaux (`<table>`) est très pratique car elle nous donne directement accès aux collections `.rows`(lignes) et `.cells` (cellules), sans avoir à se soucier des nœuds textes ou des espaces vides.

1. `table.rows` : C'est une propriété spéciale des éléments `<table>` en JS. Elle retourne une liste de tous les `<tr>`. C'est plus sûr que `table.children` car ça ignore les balises qui ne sont pas des lignes.
2. `row.cells` : De même, c'est une propriété des `<tr>` qui retourne la liste des `<td>` (ou `<th>`).
3. Au lieu de faire deux boucles imbriquées (une boucle pour les lignes, une boucle à l'intérieur pour les cellules) et de vérifier if (ligne == colonne), on fait une seule boucle. On sait déjà que la cellule cible se trouve à l'index `i` de la ligne `i`.

Voici le code :

JavaScript

```

1 <html>
2 <body>
3   <table border="1">
4     <tr>
5       <td>1</td>
6       <td>2</td>
7       <td>3</td>
8     </tr>
9     <tr>
10      <td>4</td>
11      <td>5</td>
12      <td>6</td>
13    </tr>
14    <tr>
15      <td>7</td>
16      <td>8</td>
17      <td>9</td>
18    </tr>
19  </table>
20 </body>
21 </html>
22 <script>
23   // 1. On sélectionne le tableau
24   let table = document.body.firstChild;
25
26   // 2. On parcourt les lignes du tableau
27   for (let i = 0; i < table.rows.length; i++) {
28
29     // 3. On récupère la ligne actuelle
30     let row = table.rows[i];
31
32     // 4. On récupère la cellule diagonale
33     let td = row.cells[i];
34
35     // 5. On applique la couleur (comme demandé dans l'énoncé)
36     td.style.backgroundColor = 'red';
37   }
38 </script>

```

1	2	3
4	5	6
7	8	9

Recherches dans le DOM : méthodes get et query

JavaScript met à disposition plusieurs méthodes pour rechercher et sélectionner des éléments dans le DOM. Ces méthodes permettent de cibler précisément des éléments HTML afin de les manipuler ou d'en récupérer des informations.

Méthodes getElementBy...

Les méthodes de type getElementBy... permettent de sélectionner des éléments selon des critères précis :

- getElementById() : sélection d'un élément unique à partir de son identifiant.
- getElementsByClassName() : sélection d'une liste d'éléments partageant la même classe.
- getElementsByTagName() : sélection de tous les éléments ayant le même nom de balise.

Ces méthodes retournent soit un élément unique, soit une collection d'éléments.

Méthodes querySelector et querySelectorAll

Les méthodes `querySelector` et `querySelectorAll` permettent d'utiliser des sélecteurs CSS pour rechercher des éléments dans le DOM.

- `querySelector()` : retourne le premier élément correspondant au sélecteur.
- `querySelectorAll()` : retourne tous les éléments correspondant au sélecteur sous forme de liste.

Ces méthodes sont plus flexibles car elles utilisent la même syntaxe que le CSS.

exercice 4 realiser:

Recherche d'éléments

Voici le document avec le tableau et formulaire

Comment trouver ?...

1. Le tableau avec `id="age-table"`.
2. Tous les éléments `label` dans ce tableau (il devrait y en avoir 3).
3. Le premier `td` dans ce tableau (avec le mot "Age").
4. Le `form` avec `name="search"`.
5. Le premier `input` dans ce formulaire.
6. Le dernier `input` dans ce formulaire.

lien du doc : <http://btssiogap.com/attachments/1199>

1. Le tableau avec `id="age-table"`

C'est le plus simple, on utilise l'identifiant unique.

```
let table = document.getElementById('age-table');
```

2. Tous les éléments `label` dans ce tableau

Ici, on part de la variable `table` (définie juste avant) pour ne chercher que à l'intérieur de celle-ci, et non dans toute la page.

```
let labels = table.querySelectorAll('label');
```

3. Le premier `td` dans ce tableau (avec le mot "Age")

Comme on veut le tout premier, `querySelector` est parfait car il s'arrête dès qu'il trouve une correspondance.

```
let tdAge = table.querySelector('td');
```

4. Le `form` avec `name="search"`

Il y a deux façons principales de faire cela. La méthode moderne avec un sélecteur CSS d'attribut est souvent préférée.

Méthode 1 (Sélecteur CSS d'attribut)

```
let form = document.querySelector('form[name="search"]');
```

Méthode 2 (Collection spécifique aux formulaires)

```
let form = document.forms['search'];
```

5. Le premier **input** dans ce formulaire

On cherche à l'intérieur de la variable form récupérée à l'étape précédente.

```
let firstInput = form.querySelector('input');
```

6. Le dernier **input** dans ce formulaire

C'est un peu plus subtil car il n'existe pas de sélecteur "last" direct simple en CSS standard qui fonctionne partout comme on le voudrait en JS. L'approche classique est de tous les récupérer et de prendre le dernier de la liste.

1. On récupère tous les inputs du formulaire

```
let allInputs = form.querySelectorAll('input');
```

2. On prend le dernier élément du tableau (longueur - 1)

```
let lastInput = allInputs[allInputs.length - 1];
```

Dans l'exercice 6, attention : si le bouton "Search!" est une balise **<button>** et non **<input type="submit">**, il ne sera pas sélectionné par **querySelectorAll('input')**.

Propriétés de nœud : type, balise et contenu

Chaque élément du DOM est représenté par un nœud.

Les nœuds possèdent des propriétés qui permettent d'identifier leur nature, leur type et leur contenu.

Type de nœud (nodeType)

La propriété **nodeType** permet de connaître le type d'un nœud.

Les types les plus courants sont :

- nœud élément (balise HTML),
- nœud texte,
- nœud document.

Cette propriété est utile pour différencier les éléments HTML des simples contenus textuels.

Nom de la balise (nodeName / tagName)

La propriété **nodeName** permet de connaître le nom du nœud.

Pour les éléments HTML, elle correspond au nom de la balise (exemple : DIV, TABLE, TD).

La propriété **tagName** est équivalente pour les nœuds de type élément.

Contenu du nœud

Plusieurs propriétés permettent d'accéder au contenu d'un nœud :

- **textContent** : renvoie tout le texte contenu dans le nœud,

- `innerHTML` : renvoie le contenu HTML du nœud,
- `innerText` : renvoie uniquement le texte visible par l'utilisateur.

Ces propriétés permettent de lire ou modifier dynamiquement le contenu d'une page web.

Intérêt des propriétés de nœud

Les propriétés de nœud permettent :

- d'analyser la structure du DOM,
- de comprendre la différence entre balise et contenu,
- de manipuler dynamiquement le contenu d'une page web.

Ces notions sont essentielles pour le développement web côté navigateur.

exercice 5 realiser:

Compter les descendants

Qu'affiche le script ?

```

1 <html>
2
3 <body>
4   <script>
5     alert(document.body.lastChild.nodeType);
6   </script>
7 </body>
8
9 </html>
```

Le script va afficher 1.

Voici l'explication étape par étape pour comprendre pourquoi :

1. L'exécution immédiate : Le navigateur lit le fichier HTML de haut en bas. Lorsqu'il rencontre la balise `<script>`, il arrête la construction du DOM pour exécuter le code JavaScript immédiatement.
2. L'état du DOM à cet instant :
 - La balise `<body>` est ouverte.
 - La balise `<script>` vient d'être ajoutée au DOM.
 - Tout ce qui se trouve après le script (la fermeture `</body>`, les retours à la ligne suivants, etc.) n'a pas encore été lu par le navigateur.
3. Le résultat :
 - À cet instant précis, le dernier enfant connu de `body` (`document.body.lastChild`) est donc la balise `<script>` elle-même.
 - Une balise HTML est un Nœud Élément (Element Node).
 - La propriété `nodeType` d'un élément vaut toujours 1.

exercice 6 realiser:

Balise dans le commentaire

Qu'affiche ce code ?

```
1 <script>
2   let body = document.body;
3
4   body.innerHTML = "<!--" + body.tagName + "-->";
5
6   alert( body.firstChild.data ); // Qu'est ce qu'il y a ici ?
7 </script>
```

Ce code va afficher BODY.

1. `body.tagName` : En HTML standard, la propriété `tagName` renvoie toujours le nom de la balise en majuscules. Donc `body.tagName` vaut "BODY".
2. La concaténation : L'expression "" crée la chaîne de caractères "".
3. `body.innerHTML = ...` : On remplace tout le contenu de la page par cette chaîne. Le navigateur interprète cela comme un Commentaire HTML.
 - Le DOM contient maintenant un seul nœud à l'intérieur du `<body>` : un nœud de type Commentaire.
4. `body.firstChild` : Cela cible ce fameux nœud commentaire que l'on vient de créer.
5. `.data` : Pour un nœud de type Commentaire (ou Texte), la propriété `.data` renvoie le contenu à l'intérieur du nœud (sans les balises ``).

Résultat : L'alerte affiche le contenu du commentaire, c'est-à-dire BODY.

exercice 7 realiser:

Où est le "document" dans la hiérarchie ?

À quelle classe appartient le `document` ?

Quelle est sa place dans la hiérarchie DOM ?

Hérite-t-il de `Node` ou `Element`, ou peut-être de `HTMLElement` ?

1. À quelle classe appartient le document ?

L'objet document est une instance de la classe `HTMLDocument`.

2. Quelle est sa place dans la hiérarchie ?

Voici l'arbre d'héritage complet (la chaîne de prototypes), du plus précis au plus générique :

1. `HTMLDocument` (C'est la classe spécifique pour les pages web HTML)
2. `Document` (Classe générique pour tout type de document, comme XML ou SVG)
3. `Node` (C'est un nœud du DOM, tout comme les éléments ou les textes)

4. EventTarget (C'est ce qui lui permet d'écouter des événements comme click ou DOMContentLoaded)
5. Object (La base de tous les objets en JavaScript)

3. Hérite-t-il de Node ou Element ?

- Il hérite de Node : Oui, car le document est un nœud (la racine de l'arbre).
- Il n'hérite PAS de Element (ni de HTMLElement) : C'est la distinction cruciale.
 - Element décrit une balise (comme <body>, <div>, <a>).
 - Document est le conteneur global. Il ne peut pas être une balise lui-même.

Attributs et propriétés du DOM

Lorsque le navigateur charge une page HTML, il analyse le code et crée une arborescence appelée DOM (Document Object Model).

Chaque balise HTML devient un objet JavaScript, avec des propriétés accessibles directement.

Exemples vus dans le TP :

- Accès aux propriétés d'un élément (id, className, innerHTML, etc.)
- Différence entre attribut HTML et propriété DOM
- Modification dynamique du contenu d'une page

exercice 8 realiser

Obtenez l'attribut

Écrivez le code pour sélectionner l'élément avec l'attribut `data-widget-name` dans le document et pour lire sa valeur.

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4
5   <div data-widget-name="menu">Choose the genre</div>
6
7   <script>
8     /* your code */
9   </script>
10 </body>
11 </html>
```

```

1  <script>
2  |  // 1. On sélectionne l'élément grâce au sélecteur d'attribut CSS
3  let elem = document.querySelector('[data-widget-name]');
4
5  // 2. On lit la valeur de l'attribut
6  // Méthode moderne (recommandée pour les attributs data-*)
7  console.log(elem.dataset.widgetName);
8
9  // Méthode classique (fonctionne pour tout attribut)
10 console.log(elem.getAttribute('data-widget-name'));
11 </script>

```

1. Le sélecteur [attribut] : Dans `querySelector`, on utilise des crochets `[]` pour cibler un élément qui possède un attribut spécifique, quelle que soit sa valeur.
 - `[data-widget-name]` sélectionne le premier élément qui a cet attribut.
 - `div[data-widget-name="menu"]` ciblerait spécifiquement celui qui vaut "menu".
2. La propriété `dataset` (Le point important) : Les attributs commençant par `data-` sont spéciaux. JavaScript les regroupe automatiquement dans la propriété `.dataset` de l'élément.
 - Attention à la transformation du nom : `data-widget-name` devient `widgetName` (on enlève le `data-` et on passe en `camelCase`).
 - C'est plus propre et plus facile à lire que `getAttribute`.

Donc, `elem.dataset.widgetName` renverra "menu"

exercice 9 realiser

Rendre les liens externes orange

Mettez tous les liens externes en orange en modifiant leur propriété `style`.

Un lien est externe si :

- Son `href` contient `://`
- Mais ne commence pas par `http://internal.com`.

Exemple :

```
1 <a name="list">the list</a>
2 <ul>
3   <li><a href="http://google.com">http://google.com</a></li>
4   <li><a href="/tutorial">/tutorial.html</a></li>
5   <li><a href="local/path">local/path</a></li>
6   <li><a href="ftp://ftp.com/my.zip">ftp://ftp.com/my.zip</a></li>
7   <li><a href="http://nodejs.org">http://nodejs.org</a></li>
8   <li><a href="http://internal.com/test">http://internal.com/test</a></li>
9 </ul>
10
11 <script>
12   // setting style for a single link
13   let link = document.querySelector('a');
14   link.style.color = 'orange';
15 </script>
```

Voici deux méthodes pour résoudre ce problème :

Méthode 1 :

```
<script>
// 1. On sélectionne tous les liens de la page
let links = document.querySelectorAll('a');

// 2. On parcourt chaque lien un par un
for (let link of links) {

  // 3. On récupère la valeur EXACTE de l'attribut href
  let href = link.getAttribute('href');

  // Sécurité : si le lien n'a pas de href, on passe au suivant
  if (!href) continue;

  // 4. On teste les deux conditions demandées :
  // - Doit contenir "://"
  // - Ne doit PAS commencer par "http://internal.com"
  if (href.includes('://') && !href.startsWith('http://internal.com')) {

    // Si c'est bon, on change la couleur
    link.style.color = 'orange';
  }
}
</script>
```

Méthode 2

```

1 <script> // On sélectionne tous les liens externes (ceux qui contiennent "://")
2 let externalLinks = document.querySelectorAll('a[href*="//"]:not([href^="http://internal.com"])');
3 // On applique la couleur directement sur les éléments trouvés
4 externalLinks.forEach(link => link.style.color = 'orange');
5 </script>

```

pourquoi utiliser `getAttribute('href')` ? Si on utilise `link.href` sur un lien relatif comme `<a href="/tutorial">`, le navigateur va le transformer en `http://ton-site.com/tutorial`. Du coup, il contiendrait `://` et le code le colorerait en orange par erreur ! `getAttribute` lit le texte brut du HTML, ce qui évite ce problème.

Modification du document

La modification du document consiste à créer, modifier ou supprimer des éléments HTML dynamiquement à l'aide de JavaScript.

Cette manipulation du DOM permet de rendre les pages web interactives et dynamiques, sans recharger la page.

Les actions réalisées dans le TP :

- Création de nouveaux éléments avec `createElement()`
- Ajout d'éléments dans la page avec `appendChild()`
- Modification du contenu avec `innerHTML` ou `textContent`
- Modification des styles via la propriété `style`
- Suppression d'éléments avec `remove()` ou `removeChild()`

Ces méthodes permettent d'afficher des messages personnalisés directement dans la page, plus propres et plus lisibles qu'une simple boîte alert.

exercice 10 réaliser:

createTextNode vs innerHTML vs textContent

Nous avons un élément DOM vide `elem` et une chaîne de caractères `text`.

Lesquelles de ces 3 commandes feront exactement la même chose ?

1. `elem.append(document.createTextNode(text))`
2. `elem.innerHTML = text`
3. `elem.textContent = text`

La réponse est : Les commandes 1 et 3 font exactement la même chose.

Voici pourquoi :

1. `elem.append(document.createTextNode(text))`

- Cette commande crée explicitement un "Nœud Texte".
- Cela signifie que tout ce qui est dans la variable `text` sera affiché littéralement.
- Si `text` contient `<b>Coucou</b>`, l'utilisateur verra les chevrons et les balises écrites à l'écran. Le navigateur **n'interprète pas** le HTML.

3. `elem.textContent = text`

- Cette propriété a pour but de définir le "contenu textuel".
- Elle fonctionne exactement comme la méthode 1 : elle prend la chaîne et l'insère en tant que texte brut.
- Elle est plus sûre car elle empêche l'injection de code HTML involontaire.

Pourquoi la commande 2 (`innerHTML`) est différente ?

La commande `elem.innerHTML = text` demande au navigateur de lire et interpréter la chaîne `text` comme du code HTML.

Exemple concret pour comprendre la différence :

Imaginons que `text = "<b>Salut</b>"`.

- Avec les commandes 1 et 3 :
Le DOM contiendra un nœud texte.
L'utilisateur verra écrit à l'écran : `<b>Salut</b>` (avec les caractères `<` et `>`).
- Avec la commande 2 (`innerHTML`) :
Le navigateur va comprendre "Ah, c'est une balise gras !".
Le DOM contiendra un élément `<b>`.
L'utilisateur verra écrit à l'écran : Salut (en gras, sans les balises visible).

exercice 11 réaliser:

Effacer l'élément

Créez une fonction `clear(elem)` qui supprime tout de l'élément.

```
1 <ol id="elem">
2   <li>Hello</li>
3   <li>World</li>
4 </ol>
5
6 <script>
7   function clear(elem) { /* votre code */ }
8
9   clear(elem); // efface la liste
10 </script>
```

Voici les deux solutions possibles pour la fonction `clear(elem)` :

1. La méthode

On dit simplement au navigateur : "remplace tout le contenu HTML par... rien du tout".

```
2 function clear(elem) {
3   elem.innerHTML = '';
4 }
```

2. La méthode


```

2  function clear(elem) {
3      while (elem.firstChild) {
4          elem.removeChild(elem.firstChild);
5      }
6  }

```

innerHTML = " : Le navigateur est très optimisé pour faire ça rapidement.

La boucle while : Elle est utile si tu as besoin de faire des opérations de nettoyage spécifiques sur chaque élément avant de le supprimer (par exemple, supprimer des écouteurs d'événements pour éviter des fuites de mémoire), mais pour un simple effacement visuel, c'est souvent excessif.

exercice 12 réaliser:

Pourquoi "aaa" reste-t-il ?

Dans l'exemple ci-dessous, l'appel `table.remove()` supprime le tableau du document.

mais si vous l'exécutez, vous pouvez voir que le texte "aaa" est toujours visible.

Pourquoi cela se produit-il ?

```

1  <table id="table">
2      aaa
3      <tr>
4          <td>Test</td>
5      </tr>
6  </table>
7
8  <script>
9      alert(table); // la table, comme il se doit
10
11     table.remove();
12     // pourquoi y a-t-il encore "aaa" dans le document ?
13 </script>

```

Le navigateur a sorti le texte "aaa" du tableau avant même que le script ne s'exécute.

1. La syntaxe HTML

En HTML, une balise `<table>` a une structure très stricte. Elle ne peut contenir directement que des balises spécifiques comme `<thead>`, `<tbody>`, `<tr>`, `<caption>`, etc. on a écrit le texte "aaa" directement à la racine de `<table>`, sans le mettre dans une cellule (`<td>`). C'est du **HTML invalide**.

2. L'autocorrection du navigateur ("Hoisting")

Lorsque le navigateur lit le code pour construire l'arbre DOM, il se rend compte que "aaa" n'a rien à faire là. Pour essayer de rendre la page lisible malgré l'erreur, il applique un mécanisme de correction standard :

- Il prend le texte "intrudus" ("aaa").
- Il l'éjecte **hors** du tableau.
- Il le place généralement **juste avant** la balise `<table>` dans le DOM.

3. Ce que voit le JavaScript

Au moment où le script s'exécute, le DOM ressemble en réalité à ceci :

```

2  aaa
3  <table id="table">
4    <tbody>
5      <tr>
6        <td>Test</td>
7      </tr>
8    </tbody>
9  </table>

```

The diagram illustrates the DOM structure. The text 'aaa' is shown as a sibling of the table element. The table element is represented by a box containing its children: a tbody element, which contains a tr element, which in turn contains a td element with the text 'Test'. The text 'any' is shown as a sibling of the table element, positioned between the table and its closing tag. The text 'any' is highlighted with a yellow box and a yellow star.

4. La suppression

Quand on exécute `table.remove()`, on supprime l'élément `<table>`. Mais comme "aaa" est devenu un **frère précédent** (previous sibling) du tableau et non plus un enfant, il n'est pas affecté par la suppression. Il reste donc affiché à l'écran.

exercice 13 realiser:

Créer une liste

Écrivez une interface pour créer une liste à partir des entrées utilisateur.

Pour chaque élément de la liste :

1. Interrogez un utilisateur sur son contenu en utilisant `prompt`.
2. Créez le `<li>` avec et ajoutez-le à `<ul>`.
3. Continuez jusqu'à ce que l'utilisateur annule l'entrée (en appuyant sur la touche `Esc` ou une entrée vide).

Tous les éléments doivent être créés dynamiquement.

Si un utilisateur tape des balises HTML, elles doivent être traitées comme un texte.

Voici le code complet et commenté pour répondre à cet exercice :

```

2 // 1. On crée d'abord la liste <ul> et on l'ajoute à la page
3 let ul = document.createElement('ul');
4 document.body.append(ul);
5
6 while (true) {
7   // 2. On demande à l'utilisateur de saisir quelque chose
8   let data = prompt("Entrez le contenu de votre élément de liste", "");
9
10  // 3. Condition de sortie : Si l'utilisateur annule (null) ou laisse vide ("" )
11  if (!data) {
12    break; // On casse la boucle
13  }
14
15  // 4. On crée l'élément <li>
16  let li = document.createElement('li');
17
18  // 5. IMPORTANT : On utilise textContent pour la sécurité
19  // Cela garantit que si l'utilisateur tape "<b>gras</b>", on verra les balises écrites et non du texte en gras.
20  li.textContent = data;
21
22  // 6. On attache le li à la liste parent ul
23  ul.append(li);
24 }

```

La boucle while(true) : C'est le standard pour créer une boucle infinie qui ne s'arrête que sur une condition précise (le break).

La condition if (!data) :

- Si l'utilisateur clique sur Annuler (Esc), prompt renvoie null.
- Si l'utilisateur clique sur OK sans rien écrire, prompt renvoie une chaîne vide "".
- En JavaScript, null et "" sont tous les deux considérés comme false dans un test booléen. Donc !data couvre les deux cas d'arrêt demandés.

exercice 14 réaliser:

Créer un arbre à partir de l'objet

Écrivez une fonction `createTree` qui crée une liste imbriquée `ul/li` à partir de l'objet imbriqué.

Voici la solution propre utilisant les méthodes de création du DOM

```

1 let data = {
2   "fish": {
3     "trout": {},
4     "salmon": {}
5   },
6   "trees": {
7     "redwood": {},
8     "oak": {},
9     "maple": {}
10  }
11 };
12 function createTree(container, data) {
13   let ul = document.createElement('ul');
14   for (let key in data) {
15     let li = document.createElement('li');
16     li.textContent = key;
17     if (Object.keys(data[key]).length > 0) {
18       createTree(li, data[key]);
19     }
20     ul.appendChild(li);
21   }
22   container.appendChild(ul);
23 }
24 createTree(document.body, data);

```

Résultat de l'arborescence :

- Fish
 - trout
 - salmon
- Tree
 - Huge
 - sequoia
 - oak
 - Flowering
 - apple tree
 - magnolia

exercice 15 réaliser:

Afficher les descendants dans un arbre

```
// On récupère tous les <li> de la page
let lis = document.getElementsByTagName('li');

for (let li of lis) {
  // L'astuce est ici : on cherche les 'li' À L'INTÉRIEUR du 'li' actuel
  // getElementsByTagName renvoie tous les descendants, pas juste les enfants directs
  let count = li.getElementsByTagName('li').length;

  // S'il y a des enfants, on modifie le texte
  if (count > 0) {
    // On modifie le premier nœud texte (le nom de l'élément)
    // On évite li.innerHTML ou li.textContent pour ne pas écraser la balise <ul> interne
    li.firstChild.data += ' [' + count + ']';
  }
}
```

- fish [2]
 - trout
 - salmon
- trees [3]
 - redwood
 - oak
 - maple

exercice 16 réaliser:

créer un calendrier

L'objet Date : En JS, les mois commencent à 0 (Janvier = 0, Septembre = 8). Mais l'input utilisateur est "humain" (1-12). Il faut penser à faire -1.

Le jour de la semaine : date.getDay() renvoie 0 pour Dimanche, 1 pour Lundi, etc. Or, le calendrier doit commencer par Lundi. Il faut une petite formule pour décaler ça.

Les cases vides : Si le mois commence un Jeudi, il faut insérer des cases vides (<td></td>) pour Lundi, Mardi, Mercredi.

```

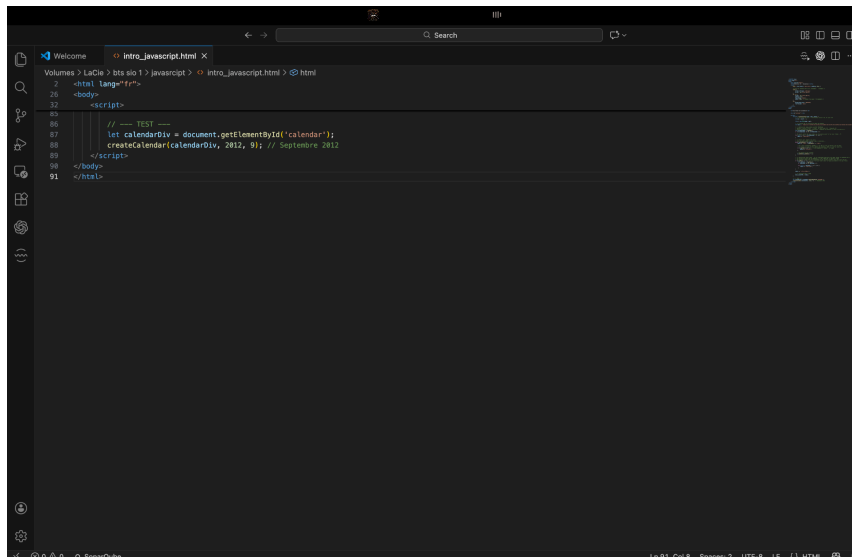
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>Exercice 25 : Calendrier</title>
6   <style>
7     body { font-family: sans-serif; padding: 20px; }
8
9     /* Style du tableau pour qu'il ressemble à l'exemple */
10    table {
11      border-collapse: collapse;
12      border: 1px solid #black;
13    }
14    th, td {
15      border: 1px solid #black;
16      padding: 5px;
17      text-align: center;
18      width: 30px; /* Largeur fixe pour l'alignement */
19    }
20    th {
21      background-color: #e6e6ed;
22      font-weight: bold;
23    }
24  </style>
25 </head>
26 <body>
27
28   <h2>Générateur de Calendrier</h2>
29
30   <div id="calenda"></div>
31
32   <script>
33     function createCalendar(elev, year, month) {
34       // 1. Correction du mois : l'utilisateur donne 1-12, JS veut 0-11
35       let mon = month - 1;
36
37       let d = new Date(year, mon);
38
39       // 2. Création de la structure de base du tableau
40       let table = "table<tr><th></th><th></th><th></th><th></th><th></th><th></th><th></th><th></th><tr>";
41
42       // 3. Gestion des cases vides au début (Padding)
43       // getDay() renvoie 0 (Dim) à 6 (Sam). On veut Lundi (0) à Dimanche (6)
44       // Astuce mathématique pour transformer 1 (Dimanche) à 0 (Lundi) : (d.getDay() + 7) % 7
45       let firstDayIndex = d.getDay();

```

```

46       let firstDayIndex = d.getDay();
47       if (firstDayIndex == 0) firstDayIndex = 7;
48
49       // On ajoute autant de cases vides que nécessaire avant le 1er jour (index - 1)
50       for (let i = 0; i < firstDayIndex - 1; i++) {
51         table += "<td></td>";
52       }
53
54       // 4. Boucle pour remplir les jours
55       // Tant que nous sommes toujours dans le bon mois...
56       while (d.getMonth() === mon) {
57         table += "<td>" + d.getDate() + "</td>";
58
59         // Si c'est un dimanche (getDay() == 0) ou si c'est le dernier jour du mois
60         // on ferme la ligne et on ouvre une nouvelle, SAUF si c'est la toute fin.
61         if (d.getDay() == 0 || (d.getMonth() + 1 < mon + 1)) {
62           table += "</tr><tr>";
63         }
64
65         // On passe au jour suivant
66         d.setDate(d.getDate() + 1);
67       }
68
69       // 5. Gestion des cases vides à la fin (optionnel mais plus joli pour fermer le tableau carré)
70       // Si le mois ne finit pas un dimanche, il faut compléter la dernière ligne.
71       // On reprend la date (qui est maintenant le 1er du mois suivant) pour voir où on en est
72       if (d.getDay() != 1) { // Si ce n'est pas lundi (donc la ligne précédente n'est pas finie)
73         let dayIndex = d.getDay();
74         if (dayIndex == 0) dayIndex = 7;
75         for (let i = dayIndex; i <= 7; i++) {
76           table += "<td></td>";
77         }
78       }
79
80       table += "</tr></table>";
81
82       // 6. Injection dans le DOM
83       elev.innerHTML = table;
84     }
85   </script>

```



```
1 <html lang="fr">
2 <body>
3   <script>
4     // --- TEST ---
5     let calendarDiv = document.getElementById('calendar');
6     createCalendar(calendarDiv, 2012, 9); // Septembre 2012
7   </script>
8 </body>
9 </html>
```

Générateur de Calendrier

MO	TU	WE	TH	FR	SA	SU
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30

Explication rapide du code

- `d.getDay()` : C'est la clé.
 - En JS standard : Dim=0, Lun=1, Mar=2...
 - L'énoncé veut Lundi en premier. J'ai donc utilisé `if (day === 0) day = 7` pour considérer Dimanche comme le 7ème jour, ce qui rend le calcul des cases vides (boucle `for`) beaucoup plus simple.
- `d.setDate(d.getDate() + 1)` : C'est comme ça qu'on avance dans le temps. L'objet `Date` est intelligent : si on est le 30 septembre et qu'on ajoute 1 jour, il passe automatiquement au 1er octobre (et change le mois). C'est pour ça que la boucle `while (d.getMonth() !== mon)` fonctionne parfaitement pour s'arrêter à la fin du mois.

exercice 17 réaliser:

horloge coloré

15 : 54 : 06

Start

Stop

```

1  <!DOCTYPE html>
2  <html lang="fr">
3  <head>
4      <meta charset="UTF-8">
5      <title>Horloge Colorée</title>
6      <style>
7          /* On gère les couleurs ici (CSS) comme demandé */
8          .hour { color: red; }
9          .min { color: green; }
10         .sec { color: blue; }
11
12         .clock {
13             font-size: 2em;
14             font-family: monospace; /* Pour que les chiffres ne bougent pas */
15             font-weight: bold;
16             margin-bottom: 10px;
17         }
18     </style>
19 </head>
20 <body>
21
22     <div class="clock">
23         <span class="hour" id="hours">00</span>:<span class="min" id="minutes">00</span>:<span class="sec" id="seconds">00</span>
24     </div>
25
26     <button onclick="clockStart()">Start</button>
27     <button onclick="clockStop()">Stop</button>
28
29     <script>
30         let timerId; // Variable pour stocker l'identifiant du timer
31
32         function update() {
33             let date = new Date(); // On prend l'heure actuelle
34
35             let hours = date.getHours();
36             // Astuce : On ajoute un '0' devant si le chiffre est < 10
37             if (hours < 10) hours = '0' + hours;
38             document.getElementById('hours').textContent = hours;
39
40             let minutes = date.getMinutes();
41             if (minutes < 10) minutes = '0' + minutes;
42             document.getElementById('minutes').textContent = minutes;
43
44             let seconds = date.getSeconds();
45             if (seconds < 10) seconds = '0' + seconds;

```

```

45             if (seconds < 10) seconds = '0' + seconds;
46             document.getElementById('seconds').textContent = seconds;
47         }
48
49         function clockStart() {
50             // Sécurité : on ne lance pas le timer s'il tourne déjà
51             if (!timerId) {
52                 // On met à jour tout de suite (pour ne pas attendre 1 seconde)
53                 update();
54                 // On lance la répétition toutes les 1000ms (1 seconde)
55                 timerId = setInterval(update, 1000);
56             }
57         }
58
59         function clockStop() {
60             // On arrête le timer grâce à son ID
61             clearInterval(timerId);
62             timerId = null; // On remet à null pour pouvoir relancer plus tard
63         }
64
65         // Optionnel : Lancer l'horloge automatiquement au chargement
66         // clockStart();
67     </script>
68 </body>
69 </html>

```

exercice 18 réaliser:

Insérer 2 et 3 entre 1 et 4

L'objectif est d'insérer le code HTML `<li>2</li><li>3</li>` exactement entre le `<li>1</li>` et le `<li>4</li>`.

La Solution (`insertAdjacentHTML`)

C'est la méthode reine pour ce genre de tâche. Elle est beaucoup plus performante que `innerHTML` (qui effacerait et recréerait toute la liste) et plus rapide à écrire que `createElement` pour plusieurs éléments d'un coup.

```
1 <script>
2   // 1. On sélectionne l'élément de référence (le "1")
3   let one = document.getElementById('one');
4
5   // 2. On insère le HTML "après la fin" de cet élément (donc juste après sa fermeture </li>)
6   one.insertAdjacentHTML('afterend', '<li>2</li><li>3</li>');
7 </script>
```

Pourquoi `afterend` ? La méthode `insertAdjacentHTML` prend 4 positions possibles par rapport à l'élément sélectionné (`one`) :

1. `'beforebegin'` : Juste avant l'ouverture `<li>` (Devenant le frère précédent).
2. `'afterbegin'` : À l'intérieur, au tout début (Premier enfant).
3. `'beforeend'` : À l'intérieur, à la toute fin (Dernier enfant).
4. `'afterend'` : Juste après la fermeture `</li>` (Frère suivant). -> C'est ce qu'on veut

exercice 19 réaliser:

Trier le tableau par nom

L'objectif est de trier les lignes du tableau (`<tr>`) en fonction de l'ordre alphabétique de la première colonne ("Name").

La Solution JavaScript

L'astuce principale ici est que les nœuds du DOM sont des objets uniques. Si tu les "insères" à nouveau quelque part (`append`), ils sont déplacés de leur ancienne position vers la nouvelle. Pas besoin de les supprimer manuellement.

la différence entre `Collection` (ce que rend le DOM) et `Array` (Tableau JS).

- `tbody.rows` est une `HTMLCollection`. Elle n'a pas la méthode `.sort()`.
- C'est pour ça qu'on doit faire `Array.from(tbody.rows)` avant de pouvoir trier. C'est une erreur très fréquente en examen !

Tableau à trier

Le script va trier automatiquement ce tableau par nom (Ordre : Ann, John, Pete).

Name	Surname	Age
Ann	Lee	5
John	Smith	10
Pete	Brown	15

```
1  <!DOCTYPE html>
2  <html lang="fr">
3  <head>
4    <meta charset="UTF-8">
5    <title>Exercice : Trier le tableau</title>
6    <style>
7      table { border-collapse: collapse; margin-bottom: 20px; }
8      th, td { border: 1px solid black; padding: 5px 10px; }
9      th { background-color: #eee; cursor: pointer; }
10   </style>
11 </head>
12 <body>
13
14   <h3>Tableau à trier</h3>
15   <p>Le script va trier automatiquement ce tableau par nom (Ordre : Ann, John, Pete).</p>
16
17   <table id="grid">
18     <thead>
19       <tr>
20         <th>Name</th>
21         <th>Surname</th>
22         <th>Age</th>
23       </tr>
24     </thead>
25     <tbody>
26       <tr>
27         <td>John</td>
28         <td>Smith</td>
29         <td>10</td>
30       </tr>
31       <tr>
32         <td>Pete</td>
33         <td>Brown</td>
34         <td>15</td>
35       </tr>
36       <tr>
37         <td>Ann</td>
38         <td>Lee</td>
39         <td>5</td>
40       </tr>
41     </tbody>
42   </table>
43
44   <script>
45     // --- SOLUTION ---
46
```

```

45 // --- SOLUTION ---
46
47 let table = document.getElementById('grid');
48 let tbody = table.querySelector('tbody');
49
50 // 1. Convertir la collection de lignes en vrai tableau JS
51 let rowsArray = Array.from(tbody.rows);
52
53 // 2. Trier
54 rowsArray.sort((rowA, rowB) => {
55     return rowA.cells[0].textContent.localeCompare(rowB.cells[0].textContent);
56 });
57
58 // 3. Réinsérer (ce qui déplace les éléments)
59 tbody.append(...rowsArray);
60 </script>
61
62 </body>
63 </html>

```

Styles et classes

En JavaScript, il est possible de modifier l'apparence des éléments HTML en agissant sur leurs styles ou leurs classes CSS.

Il existe deux façons principales de styliser un élément :

1. Modifier directement le style avec la propriété style
2. Ajouter ou supprimer des classes CSS, méthode à privilégier

Modification directe des styles

La propriété style permet de modifier un style directement depuis JavaScript.

Exemples :

- Changement de couleur
- Déplacement d'un élément
- Modification de la taille ou de la position

Cette méthode est surtout utilisée lorsque les valeurs sont calculées dynamiquement (ex : position d'un élément).

Utilisation des classes CSS

Il est recommandé d'utiliser des classes CSS plutôt que style, car cela rend le code plus lisible et plus facile à maintenir.

Deux propriétés ont été utilisées :

- className
- classList

Avec classList, il est possible de :

- add() une classe

- `remove()` une classe
- `toggle()` une classe
- `contains()` vérifier la présence d'une classe

Cette méthode permet de modifier l'apparence d'un élément simplement en lui appliquant une classe définie en CSS.

Exercice 20 : Créer une Notification

L'objectif est de créer une fonction `showNotification(options)` qui affiche un message flottant à une position précise et le fait disparaître automatiquement après 1,5 seconde.

`setTimeout` : C'est la fonction standard pour différer une action. Ici, on s'en sert pour nettoyer le DOM en supprimant l'élément une fois le temps écoulé.

```
JS alert.js > ...
1 function showNotification(options) {
2   // 1. Créer l'élément <div>
3   let notification = document.createElement('div');
4
5   // 2. Ajouter la classe de base "notification"
6   notification.classList.add('notification');
7
8   // 3. Ajouter la classe optionnelle si elle existe
9   if (options.className) {
10    notification.classList.add(options.className);
11  }
12
13  // 4. Définir le contenu HTML
14  notification.innerHTML = options.html;
15
16  // 5. Appliquer le style CSS (positionnement)
17  // On utilise "fixed" pour que ça reste à l'écran même si on scrolle
18  notification.style.position = 'fixed';
19
20  // On gère les valeurs par défaut (0px si non défini) et on ajoute "px"
21  notification.style.top = (options.top || 0) + 'px';
22  notification.style.right = (options.right || 0) + 'px';
23
24  // 6. Insérer l'élément dans le corps de la page (body)
25  document.body.append(notification);
26
27  // 7. Faire disparaître la notification après 1.5 secondes (1500 ms)
28  setTimeout(() => {
29    notification.remove();
30  }, 1500);
31 }
32 showNotification({
33   top: 10,
34   right: 10,
35   html: "Hello!",
36   className: "welcome"
37 });
```

Taille des éléments et défilement

JavaScript permet de mesurer et de contrôler la taille des éléments ainsi que la position du défilement sur la page. Ces notions sont utiles pour créer des interfaces dynamiques et réactives.

Mesurer la taille d'un élément

Plusieurs propriétés du DOM permettent d'obtenir les dimensions :

- `offsetWidth / offsetHeight` : largeur et hauteur incluant les bordures
- `clientWidth / clientHeight` : largeur et hauteur sans les bordures, mais avec le padding
- `scrollWidth / scrollHeight` : largeur et hauteur totale du contenu, même si une partie est invisible à l'écran

Contrôle du défilement

On peut lire et modifier la position du scroll de la page ou d'un élément spécifique.

Propriétés importantes :

- `scrollTop / scrollLeft` : position verticale/horizontale du défilement
- `scrollBy(x, y)` : déplace le scroll de manière relative
- `scrollTo(x, y)` : déplace le scroll à une position absolue

Applications pratiques :

- Faire défiler automatiquement une page ou un élément
- Créer des effets d'animation lors du scroll
- Vérifier si un utilisateur a atteint le bas d'une section ou de la page

Exercice 21 défilement vers le bas :

Le concept de "défilement depuis le bas" (`scrollTop`) n'existe pas nativement en JavaScript, contrairement à `scrollTop`. Il faut donc le calculer mathématiquement.

La Logique

Imagine ton élément comme une longue page déroulante :

1. `elem.scrollHeight` : C'est la hauteur totale du contenu (y compris la partie cachée).
2. `elem.scrollTop` : C'est la partie cachée en haut (ce que tu as déjà fait défiler).
3. `elem.clientHeight` : C'est la hauteur visible de l'élément (la fenêtre à travers laquelle tu regardes).

Pour savoir ce qu'il reste en bas, il suffit de faire une soustraction :

$\text{Reste} = \text{Total} - (\text{Ce_qui_est_en_haut} + \text{Ce_qui_est_visible})$

```
1 <script>
2   function getScrollBottom(elem) {
3       return elem.scrollHeight - elem.scrollTop - elem.clientHeight;
4   }
5 </script>
```

Exercice 22 Calculer la largeur de la Scrollbar

L'astuce consiste à créer un élément, à forcer l'affichage d'une barre de défilement, et à comparer sa largeur totale avec sa largeur intérieure (utilisable).

$\text{Largeur_Scrollbar} = \text{Largeur_Totale}(\text{offsetWidth}) - \text{Largeur_Interne}(\text{clientWidth})$

```
<script>
function getScrollbarWidth() {

    const div = document.createElement('div');

    div.style.width = '100px';
    div.style.height = '100px';
    div.style.overflow = 'scroll';
    div.style.position = 'absolute';
    div.style.top = '-9999px';

    document.body.append(div);

    let scrollbarWidth = div.offsetWidth - div.clientWidth;

    div.remove();

    return scrollbarWidth;
}

alert("La barre de défilement fait " + getScrollbarWidth() + "px de large.");
</script>
```

offsetWidth : C'est la largeur "bord à bord", incluant les bordures et la barre de défilement.

clientWidth : C'est la largeur "interne", c'est-à-dire l'espace disponible pour le contenu (excluant la barre de défilement et les bordures).

En soustrayant les deux, on isole mathématiquement l'épaisseur de la barre.

Exercice 23 placer le ballon au centre du terrain

Pour centrer la balle, il ne suffit pas de mettre top: 50% et left: 50%, car cela placerait le coin supérieur gauche de la balle au centre. Il faut aussi décaler la balle de la moitié de sa propre taille vers le haut et vers la gauche.

Voici le code JavaScript pour réaliser cela, avec les explications détaillées.

```

<script>
// 1. On récupère les éléments du DOM
const field = document.getElementById('field');
const ball = document.getElementById('ball');

// 2. On calcule les coordonnées pour centrer la balle

// Pour la position horizontale (left) :
// (Largeur du terrain / 2) - (Largeur de la balle / 2)
let leftValue = (field.clientWidth / 2) - (ball.offsetWidth / 2);

// Pour la position verticale (top) :
// (Hauteur du terrain / 2) - (Hauteur de la balle / 2)
let topValue = (field.clientHeight / 2) - (ball.offsetHeight / 2);

// 3. On applique les valeurs calculées au style de la balle
// N'oublie pas d'ajouter l'unité 'px' !
ball.style.left = leftValue + 'px';
ball.style.top = topValue + 'px';
</script>
</body>

```

Exercice 24 différence largeur css vs clientwidth

Voici les 3 différences majeures entre `getComputedStyle(elem).width` et `elem.clientWidth` :

1. Le type de données (String vs Number)

- `getComputedStyle(elem).width` renvoie une chaîne de caractères contenant l'unité (ex: "300px"). Vous ne pouvez pas faire de calculs directs (+ 10) sans la transformer d'abord (avec `parseInt` ou `parseFloat`).
- `elem.clientWidth` renvoie un nombre entier (ex: 300). C'est prêt à l'emploi pour des calculs mathématiques.

2. La prise en compte du Padding et de la Scrollbar

- `elem.clientWidth` représente la largeur de la zone interne utile. Elle inclut le contenu + le padding, mais soustrait la barre de défilement (scrollbar) si elle est présente.
- `getComputedStyle(elem).width` renvoie la valeur résolue de la propriété CSS `width`.
 - Par défaut (avec `box-sizing: content-box`), c'est uniquement la largeur du contenu (sans le padding).
 - Elle ne soustrait pas la barre de défilement de la même manière ; c'est la largeur "telle que définie par le CSS".

3. La précision

- `elem.clientWidth` arrondit toujours la valeur à l'entier le plus proche.
- `getComputedStyle(elem).width` peut renvoyer une valeur décimale précise (ex: "300.5px").

Coordonnées des éléments

JavaScript permet de **récupérer les coordonnées d'un élément** par rapport à la page ou à la fenêtre. Cela est utile pour créer des animations, positionner des éléments dynamiquement ou gérer des interactions avec l'utilisateur.

Propriétés principales

- **offsetTop / offsetLeft** : position de l'élément par rapport à son parent positionné (offsetParent)
- **offsetHeight / offsetWidth** : dimensions de l'élément incluant les bordures
- **getBoundingClientRect()** : renvoie un objet avec les coordonnées de l'élément par rapport à la **fenêtre de visualisation** (top, left, right, bottom, width, height)

Exemples d'utilisation

- Déterminer si un élément est visible à l'écran
- Déplacer un élément en fonction de sa position
- Détecter la collision ou le survol d'un élément par un autre

Exercice 25 trouver les coordonnées de la fenêtre du champ

Pour réussir cet exercice, il faut combiner deux concepts :

1. **getBoundingClientRect()** : Cette méthode renvoie un objet contenant left, top, right, bottom, width et height de l'élément entier (y compris ses bordures et padding) par rapport à la fenêtre visible.
 - Cela nous donne directement les réponses pour les flèches 1 et 2 (l'extérieur).
2. **clientLeft / clientTop** : C'est une astuce à connaître.
 - **clientLeft** correspond à la largeur de la bordure gauche.
 - **clientTop** correspond à la largeur de la bordure haute.
 - En les ajoutant aux coordonnées de base (**rect.left** et **rect.top**), on "passe par-dessus" la bordure pour arriver à l'intérieur (flèche 3).
3. **clientWidth / clientHeight** : Comme vu dans la question précédente, cela représente la zone intérieure (contenu + padding, mais sans bordures).
 - Pour trouver le point 4, on part de l'intérieur gauche et on ajoute cette largeur interne.

Conclusion

Cet atelier a permis de maîtriser les principaux outils de manipulation du DOM dans le navigateur avec JavaScript.

Nous avons vu comment :

- Accéder et modifier les propriétés et attributs des éléments HTML.

- Rechercher des éléments efficacement avec `getElementById`, `getElementsByClassName`, `querySelector` et `querySelectorAll`.
- Modifier le contenu et la structure de la page en créant, supprimant ou changeant des éléments.
- Gérer les styles et les classes CSS pour modifier l'apparence des éléments dynamiquement.
- Mesurer la taille et les coordonnées des éléments et contrôler le défilement de la page.
- Créer des interactions avec l'utilisateur grâce aux événements (click, mouseover, etc.).

En résumé, ces notions sont essentielles pour rendre une page web interactive et dynamique, et constituent une base solide pour tous les projets web futurs.